

MEMORIA DEL PROYECTO

Implantación y Customización de Odoo

– Club Deportivo de Pádel –



ÍNDICE

MEMORIA DEL PROYECTO	1
Implantación y Customización de Odoo	1
-- Club Deportivo de Pádel --	1
1. Escenario y necesidades	2
1.1 Descripción de la organización	2
1.2 Problemas actuales	2
1.3 Cómo la customización de Odoo resuelve los problemas	2
1.4 Beneficios obtenidos	3
2. Documentación técnica	4
2.1 Fuentes consultadas	4
Documentación oficial	4
Recursos técnicos	4
Uso de herramientas de IA.....	4
2.2 Pasos realizados	5
2.3 Código y módulos creados	6
Módulo propio: club_padel	6
Modelos creados	6
Vistas XML personalizadas	6
CSS personalizado.....	7
2.4 Backup de la base de datos	7
3. Customizaciones realizadas.....	8
3.1 Modificación de campos y vistas.....	8
3.2 Gestión de clientes, proveedores y productos.....	8
3.3 Creación de módulo propio	8
3.4 Cambios en el entorno gráfico	9
3.5 Mejoras extra realizadas.....	9
4. Customizaciones realizadas.....	10
4.1 Entorno de Desarrollo	10
4.2 Arquitectura de Odoo.....	10

5. Diseño del Modelo de Datos	10
5.1 Modelo Principal: Pistas	11
5.2 Modelo de Reservas (Base Preparada).....	11
6. Seguridad y Permisos.....	11
7. Personalización del Entorno Gráfico	12
7.1 Gestión de Assets	12
8. Metodología de Trabajo	13
9. Pruebas Realizadas	13
• Pruebas funcionales	13
• Pruebas de actualización	13
• Pruebas de consistencia	13
10. Rendimiento y Optimización	13
11. Escalabilidad del Proyecto	14
12. Plan de Mantenimiento	14
13. Valor Técnico del Proyecto	15
14. Conclusión.....	15

1. Escenario y necesidades

1.1 Descripción de la organización

La organización elegida es un **club deportivo de pádel** de tamaño medio que dispone de:

- 6 pistas de pádel.
- Gestión de socios y clientes ocasionales.
- Venta de productos (palas, pelotas, camisetas).
- Organización de torneos.
- Proveedores de material deportivo.
- Gestión administrativa básica (facturación y control de clientes).

Actualmente el club trabaja con hojas de cálculo y herramientas separadas para gestionar reservas, clientes y ventas.

1.2 Problemas actuales

El club presenta los siguientes problemas:

- Información dispersa en Excel y papel.
- No existe control centralizado de clientes y proveedores.
- No hay trazabilidad clara de ventas de productos.
- Gestión manual de reservas.
- Imagen digital poco profesional.
- Dificultad para generar informes y estadísticas.

Esto provoca pérdida de tiempo, errores administrativos y falta de control financiero.

1.3 Cómo la customización de Odoo resuelve los problemas

- Se ha implementado Odoo 16 como ERP centralizado mediante el desarrollo de tres módulos propios que trabajan de forma modular e integrada:

- **club_padel** → Gestión de pistas y reservas del club.
- **tienda_padel** → Gestión de productos deportivos (palas, pelotas, camisetas).
- **club_padel_tienda** → Integración entre reservas y productos mediante un sistema de carrito por reserva.
- Esta arquitectura modular permite separar responsabilidades, mantener el código organizado y facilitar futuras ampliaciones del sistema sin afectar al núcleo principal del ERP.
- La solución aporta:
- Gestión centralizada de reservas.
- Control completo de productos de tienda.
- Relación directa entre reservas y ventas de productos.
- Arquitectura escalable basada en módulos independientes.
- Personalización del entorno gráfico adaptada al club deportivo.

1.4 Beneficios obtenidos

- Centralización de datos.
- Reducción de errores administrativos.
- Mayor control de inventario.
- Mejora de imagen corporativa.
- Escalabilidad del sistema.
- Base digital profesional para el club.

2. Documentación técnica

2.1 Fuentes consultadas

Documentación oficial

- Documentación oficial de Odoo 16 (backend, creación de módulos).
- Manual de desarrollo de módulos Odoo.

Utilidad:

- Comprensión de estructura de módulos.
- Uso de `__manifest__.py`.
- Herencia de vistas.
- Uso de assets para CSS.

Recursos técnicos

- Foros técnicos de desarrolladores Odoo.
- Ejemplos de módulos base.

Utilidad:

- Corrección de errores en carga de assets.
- Solución de problemas con actualización de módulos.
- Gestión de `web.assets_backend`.

Uso de herramientas de IA

Se utilizó asistencia de IA para:

- Corrección de errores en CSS.
- Verificación de estructura del módulo.
- Diagnóstico de problemas con assets.
- Explicación de procesos de actualización.

Ejemplo de prompt utilizado:

“¿Por qué mi archivo CSS no se carga en web.assets_backend en Odoo 16?”

Respuesta obtenida:

- Verificar `__manifest__.py`.
- Actualizar módulo con `-u nombre_modulo`.
- Reiniciar servicio.
- Usar `?debug=assets` para comprobar carga.

2.2 Pasos realizados

1. Instalación de Odoo 16 en servidor local Linux.
2. Creación de carpeta personalizada para addons:

```
operador_odoo@ldapserver:~/custom_addons/club_padel$ ls /opt/odoo/custom_addons/  
club_padel  club_padel_tienda  tienda_padel  
operador_odoo@ldapserver:~/custom_addons/club_padel$
```

3. Desarrollo de tres módulos propios:

club_padel

```
operador_odoo@ldapserver:~/custom_addons/club_padel$ sudo tree -L 2 /opt/odoo/custom_addons/club_padel
/opt/odoo/custom_addons/club_padel
├── data
│   ├── demo.xml
│   ├── productos_demo.xml
│   ├── secuencia_ventas.xml
│   ├── secuencia.xml
│   ├── tienda_data.xml
│   └── tienda_productos_demo.xml
├── __init__.py
├── __manifest__.py
├── models
│   ├── cliente_contacto.py
│   ├── __init__.py
│   ├── pedido_tienda.py
│   ├── pista_padel.py
│   ├── producto_padel.py
│   ├── __pycache__
│   ├── reserva_padel.py
│   ├── tienda_producto.py
│   └── venta_tienda.py
├── __pycache__
├── __init__.cpython-310.pyc
├── report
│   ├── plantilla_reporte_reserva.xml
│   └── reporte_reserva.xml
├── security
│   └── ir.model.access.csv
├── static
│   ├── description
│   └── src
├── views
│   ├── clientes_vistas.xml
│   ├── menu.xml
│   ├── pistas_vistas.xml
│   ├── productos_vistas.xml
│   ├── reserva_extension_tienda.xml
│   ├── reservas_vistas.xml
│   ├── tienda_productos_action.xml
│   ├── tienda_producto_views.xml
│   └── tienda_vistas.xml
10 directories, 29 files
operador_odoo@ldapserver:~/custom_addons/club_padel$
```


tienda_padel

```
operador_odoo@ldapserver:~/custom_addons/club_padel$ sudo tree -L 2 /opt/odoo/custom_addons/club_padel/tienda_padel
/opt/odoo/custom_addons/club_padel/tienda_padel
├── data
│   └── demo.xml
├── __init__.py
├── __manifest__.py
├── models
│   ├── __init__.py
│   ├── __pycache__
│   └── tienda_producto.py
├── __pycache__
│   └── __init__.cpython-310.pyc
├── security
│   └── ir.model.access.csv
├── static
│   └── description
├── views
│   ├── menu.xml
│   └── tienda_producto_views.xml
└──
```

8 directories, 9 files
operador_odoo@ldapserver:~/custom_addons/club_padel\$

club_padel_tienda

```
operador_odoo@ldapserver:~/custom_addons/club_padel$ sudo tree -L 2 /opt/odoo/custom_addons/club_padel_tienda
/opt/odoo/custom_addons/club_padel_tienda
├── __init__.py
├── __manifest__.py
├── models
│   ├── carrito.py
│   ├── __init__.py
│   └── __pycache__
├── __pycache__
│   └── __init__.cpython-310.pyc
├── security
│   └── ir.model.access.csv
├── views
│   ├── carrito_views.xml
│   └── menu.xml
└──
```

5 directories, 8 files
operador_odoo@ldapserver:~/custom_addons/club_padel\$

4. Definición de modelos personalizados en Python siguiendo la arquitectura MVC de Odoo.
5. Creación de vistas XML personalizadas (formularios, listas y menús).
6. Configuración de permisos de acceso mediante ir.model.access.csv.
7. Integración de estilos CSS personalizados en web.assets_backend.
8. Actualización de módulos mediante comandos de terminal: `odoo-bin -d bd_empresa1 -u nombre_modulo`
9. Reinicio del servicio Odoo y validación funcional de cada módulo.

2.3 Código y módulos creados

Se han desarrollado tres módulos propios para cubrir las necesidades específicas del club:

Módulo: club_padel

Responsable de la gestión principal del club deportivo:

- Gestión de pistas de pádel.
- Gestión de reservas horarias.
- Relación con clientes utilizando el modelo estándar res.partner.

Modelo de pistas (club_padel)

```
GNU nano 6.2 club_padel/models/pista_padel.py
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class PistaPadel(models.Model):
    _name = "club_padel.pista"
    _description = "Pista de pádel"
    _rec_name = "nombre"

    nombre = fields.Char(string="Nombre", required=True)
    codigo = fields.Char(string="Código", required=True)
    activa = fields.Boolean(default=True)

    tipo_pista = fields.Selection(
        [("indoor", "Indoor"), ("outdoor", "Outdoor")],
        string="Tipo de pista",
        default="indoor",
        required=True,
    )

    precio_hora = fields.Float(string="Precio por hora (€)", required=True, default=18.0)
    notas = fields.Text(string="Notas")

    _sql_constraints = [
        ("codigo_unico", "unique(codigo)", "El código de pista ya existe."),
    ]

    @api.constrains("precio_hora")
    def _comprobar_precio(self):
        for rec in self:
            if rec.precio_hora < 0:
                raise ValidationError("El precio por hora no puede ser negativo.")
```

Modelo de reservas (club_padel):

```

from odoo import models, fields, api
from odoo.exceptions import ValidationError

class ReservaPadel(models.Model):
    _name = "club_padel.reserva"
    _description = "Reserva de pista"
    _rec_name = "referencia"

    referencia = fields.Char(string="Referencia", required=True, copy=False, readonly=True, default="")

    cliente_id = fields.Many2one("res.partner", string="Cliente", required=True)
    pista_id = fields.Many2one("club_padel.pista", string="Pista", required=True)

    inicio = fields.Datetime(string="Inicio", required=True)
    fin = fields.Datetime(string="Fin", required=True)

    telefono_contacto = fields.Char(string="Teléfono", related="cliente_id.phone", readonly=True)
    email_contacto = fields.Char(string="Email", related="cliente_id.email", readonly=True)

    estado = fields.Selection(
        [
            ("borrador", "Borrador"),
            ("confirmada", "Confirmada"),
            ("realizada", "Realizada"),
            ("cancelada", "Cancelada"),
        ],
        string="Estado",
        default="borrador",
        required=True,
    )

    duracion_horas = fields.Float(string="Duración (h)", compute="_calcular_duracion", store=True)
    total = fields.Float(string="Total (€)", compute="_calcular_total", store=True)

    observaciones = fields.Text(string="Observaciones")

    @api.model
    def create(self, vals):
        if vals.get("referencia", "Nuevo") == "Nuevo":
            vals["referencia"] = self.env["ir.sequence"].next_by_code("club_padel.reserva.seq")
        return super().create(vals)

    @api.depends("inicio", "fin")
    def _calcular_duracion(self):
        for rec in self:
            if rec.inicio and rec.fin:
                delta = rec.fin - rec.inicio
                rec.duracion_horas = delta.total_seconds() / 3600.0
            else:
                rec.duracion_horas = 0

    @api.depends("duracion_horas", "pista_id.precio_hora")
    def _calcular_total(self):
        for rec in self:
            rec.total = rec.duracion_horas * (rec.pista_id.precio_hora or 0)

    @api.constrains("inicio", "fin")
    def _comprobar_fechas(self):
        for rec in self:
            if rec.inicio and rec.fin and rec.fin <= rec.inicio:
                raise ValidationError("La fecha/hora de fin debe ser posterior al inicio.")

    @api.constrains("pista_id", "inicio", "fin", "estado")
    def _comprobar_solape(self):
        for rec in self:
            if not rec.pista_id or not rec.inicio or not rec.fin:
                continue
            if rec.estado == "cancelada":
                continue

            dominio = [
                ("id", "!=", rec.id),
                ("pista_id", "=", rec.pista_id.id),
                ("estado", "!=", "cancelada"),
                ("inicio", "<", rec.fin),
                ("fin", ">", rec.inicio),
            ]
            if self.search_count(dominio) > 0:
                raise ValidationError("Ya existe una reserva que se solapa en esa pista y horario")

```

```
# --- BOTONES DEL FORMULARIO (coinciden con la vista) ---
def accion_confirmar(self):
    for rec in self:
        rec.estado = "confirmada"

def accion_realizada(self):
    for rec in self:
        rec.estado = "realizada"

def accion_cancelar(self):
    for rec in self:
        rec.estado = "cancelada"

def accion_imprimir_confirmacion(self):
    accion = self.env.ref("club_padel.accion_reporte_reserva")
    return accion.report_action(self)
```

Modelo productos tienda (tienda_padel):

```
GNU nano 6.2 tienda_padel/models/tienda_producto.py
from odoo import models, fields

class TiendaPadelProducto(models.Model):
    _name = "tienda_padel.producto"
    _description = "Producto Tienda Pádel"
    _rec_name = "nombre"

    nombre = fields.Char(string="Nombre", required=True)
    categoria = fields.Selection(
        [
            ("palas", "Palas"),
            ("pelotas", "Pelotas"),
            ("textil", "Textil"),
            ("accesorios", "Accesorios"),
        ],
        string="Categoría",
        default="accesorios",
        required=True
    )
    precio = fields.Float(string="Precio (€)", default=0.0)
    stock = fields.Integer(string="Stock", default=0)
    activo = fields.Boolean(string="Activo", default=True)
    descripcion = fields.Text(string="Descripción")
    imagen = fields.Image(string="Imagen")
```

Modelo carrito integración (club_padel_tienda):

```

GNU nano 6.2                                club_padel_tienda/models/carrito.py
from odoo import models, fields, api

class ClubPadelTiendaCarrito(models.Model):
    _name = "club_padel_tienda.carrito"
    _description = "Carrito de tienda para una reserva"
    _rec_name = "name"

    name = fields.Char(string="Carrito", compute="_compute_name", store=True)
    reserva_id = fields.Many2one("club_padel.reserva", string="Reserva", required=True, ondelete="setnull")

    linea_ids = fields.One2many(
        "club_padel_tienda.carrito_linea",
        "carrito_id",
        string="Lineas"
    )

    total = fields.Float(string="Total (€)", compute="_compute_total", store=True)

    @api.depends("reserva_id")
    def _compute_name(self):
        for rec in self:
            # No asumimos el nombre de campo de la reserva (puede ser referencia, name, etc.)
            # Mostramos "Reserva <id>" para no fallar nunca.
            rec.name = f"Carrito - Reserva {rec.reserva_id.id}" if rec.reserva_id else "Carrito"

    @api.depends("linea_ids.subtotal")
    def _compute_total(self):
        for rec in self:
            rec.total = sum(rec.linea_ids.mapped("subtotal"))

class ClubPadelTiendaCarritoLinea(models.Model):
    _name = "club_padel_tienda.carrito_linea"
    _description = "Línea de carrito"

    carrito_id = fields.Many2one("club_padel_tienda.carrito", string="Carrito", required=True, ondelete="setnull")
    producto_id = fields.Many2one("tienda_padel.producto", string="Producto", required=True)

    cantidad = fields.Float(string="Cantidad", default=1.0)
    precio = fields.Float(string="Precio (€)", related="producto_id.precio", store=True, readonly=True)
    subtotal = fields.Float(string="Subtotal (€)", compute="_compute_subtotal", store=True)

    @api.depends("cantidad", "precio")
    def _compute_subtotal(self):
        for line in self:
            line.subtotal = (line.cantidad or 0.0) * (line.precio or 0.0)

```

Vistas XML personalizadas

- Formularios personalizados.
- Listas adaptadas.
- Menús propios en el backend.
- Modificación de vistas existentes mediante herencia.

CSS personalizado

Archivo:

`static/src/css/club_padel.css`

Incluye:

- Cambio de color de navbar.
- Estilo personalizado para botones.
- Cabeceras de tabla modificadas.
- Fondo corporativo.

2.4 Backup de la base de datos

Se realizó copia de seguridad mediante:

- Odoo backend (gestor de bases de datos).
- Exportación en formato .zip.

Contraseñas necesarias:

- Usuario administrador Odoo.
- Usuario PostgreSQL.

[bd_empresa1_2026-02-24_19-14-22](#)

3. Customizaciones realizadas

3.1 Modificación de campos y vistas

- Añadido campo “Estado” en modelo de pistas.
- Modificación de formularios.
- Cambio de etiquetas.
- Organización visual mejorada.
- Personalización de listas y filtros.

```
operador_odoo@ldapserver:~/custom_addons$ grep -n "fields" /opt/odoo/custom_addons/  
club_padel/models/pista_padel.py  
1:from odoo import models, fields, api  
9:     nombre = fields.Char(string="Nombre", required=True)  
10:     codigo = fields.Char(string="Código", required=True)  
11:     activa = fields.Boolean(default=True)  
13:     tipo_pista = fields.Selection(  
20:     precio_hora = fields.Float(string="Precio por hora (€)", required=True, default=18.0)  
21:     notas = fields.Text(string="Notas")  
operador_odoo@ldapserver:~/custom_addons$
```

3.2 Gestión de clientes, proveedores y productos

Se utilizaron los módulos estándar:

- Contacts (clientes y proveedores).
- Sales.
- Inventory.
- Products.

Se adaptaron para:

- Clientes socios del club.
- Proveedores de material deportivo.
- Productos específicos de pádel.

3.3 Creación de módulo propio

Se desarrollaron tres módulos personalizados adaptados al entorno del club deportivo:

- **club_padel**: módulo principal para la gestión de pistas y reservas.

- **tienda_padel:** módulo específico para la gestión de productos deportivos.
- **club_padel_tienda:** módulo puente que integra reservas con productos mediante un sistema de carrito asociado.

Esta estructura modular evita dependencias circulares entre módulos y permite activar o desactivar funcionalidades sin afectar al resto del sistema, cumpliendo las buenas prácticas de desarrollo en Odoo.

`club_padel/__manifest__.py`


```
{
    "name": "Club Pádel",
    "version": "16.0.1.0.0",
    "category": "Services",
    "summary": "Gestión de pistas, reservas y tienda para un club de pádel",
    "author": "Rubén y Ángel",
    "depends": ["base", "contacts", "product", "web"],
    "data": [
        # =====
        # SEGURIDAD
        # =====
        "security/ir.model.access.csv",

        # =====
        # DATOS / DEMO / SECUENCIAS
        # =====
        "data/secuencia.xml",
        "data/secuencia_ventas.xml",
        "data/demo.xml",
        "data/productos_demo.xml",
        "data/tienda_productos_demo.xml",

        # =====
        # REPORTES
        # =====
        "report/reporte_reserva.xml",

        # =====
        # VISTAS PRINCIPALES
        # =====
        "views/pistas_vistas.xml",
        "views/reservas_vistas.xml",
        "views/clientes_vistas.xml",
        "views/productos_vistas.xml",

        # =====
        # TIENDA (NUEVO)
        # =====
        "views/tienda_producto_views.xml",
        "views/tienda_productos_action.xml",

        # =====
        # MENÚ (SIEMPRE AL FINAL)
        # =====
        "views/menu.xml",
    ],
    "assets": {
        "web.assets_backend": [
            "club_padel/static/src/css/club_padel.css",
        ],
    },
    "application": True,
    "installable": True,
    "license": "LGPL-3",
}
```

tienda_padel/_manifest__.py

```
GNU nano 6.2 /opt/odoo/custom addons/tienda_padel/_manifest_.py
{
    "name": "Tienda Pádel",
    "version": "16.0.1.0.0",
    "category": "Sales",
    "summary": "Gestión de productos para una tienda de pádel",
    "author": "Rubén y Ángel",
    "depends": ["base", "web"],
    "data": [
        "security/ir.model.access.csv",
        "views/tienda_producto_views.xml",
        "views/menu.xml",
        "data/demo.xml",
    ],
    "application": True,
    "installable": True,
    "license": "LGPL-3",
}
```

club_padel_tienda/__manifest__.py

```
GNU nano 6.2 /opt/odoo/custom addons/club_padel_tienda/_manifest_.py
{
    "name": "Club Pádel - Tienda (Integración)",
    "version": "16.0.1.0.0",
    "category": "Services",
    "summary": "Conecta reservas del club con productos de la tienda",
    "author": "Rubén y Ángel",
    "depends": ["base", "club_padel", "tienda_padel"],
    "data": [
        "security/ir.model.access.csv",
        "views/carrito_views.xml",
        "views/menu.xml",
    ],
    "application": False,
    "installable": True,
    "license": "LGPL-3",
}
```

Funcionalidades:

- Gestión de pistas.
- Modelo personalizado.
- Menú propio en el backend.
- Integración con clientes.
- Estética personalizada.

3.4 Cambios en el entorno gráfico

Se realizaron cambios en:

- Color del navbar.

- Botones primarios.
- Fondo del sistema.
- Cabeceras de tabla.
- Identidad visual adaptada al club.

Se modificaron los assets mediante:

```
"assets": {
  "web.assets_backend": [
    "club_padel/static/src/css/club_padel.css",
  ],
}
```

3.5 Mejoras extra realizadas

- Uso de herencia de vistas.
- Diagnóstico avanzado de assets.
- Preparación para futuras funcionalidades:
 - Sistema completo de reservas.
 - Control de torneos.
 - Paneles de estadísticas.
 - Dashboard financiero.

3.6 Mejoras extra realizadas

Se implementó un módulo adicional denominado **club_padel_tienda** que actúa como puente entre los módulos club_padel y tienda_padel.

Funcionalidades principales:

- Asociar productos de la tienda a una reserva concreta.
- Crear carritos vinculados a reservas.
- Calcular automáticamente el total de productos seleccionados.

- Mantener independencia entre módulos mediante relaciones Many2one y One2many.

Esta integración permite simular un flujo real de negocio en el club, donde un cliente puede reservar una pista y adquirir productos deportivos en el mismo proceso operativo.

4. Customizaciones realizadas

4.1 Entorno de Desarrollo

La implantación se realizó en un entorno Linux con:

- Ubuntu Server
- Odoo 16 Community
- PostgreSQL como motor de base de datos
- Python 3.10
- Servidor ejecutándose en puerto 8069

Se utilizó estructura estándar de Odoo con carpeta personalizada:

`/opt/odoo/custom_addons/`

Esto permite:

- Separar código propio del core.
- Facilitar mantenimiento.
- Evitar conflictos en actualizaciones futuras.

4.2 Arquitectura de Odoo

Odoo funciona bajo arquitectura MVC:

- **Modelo (Models):** Python → definición de datos.
- **Vista (Views):** XML → representación visual.
- **Controlador:** Gestionado internamente por Odoo ORM.

El módulo club_padel respeta esta estructura modular, lo que garantiza:

- Escalabilidad
- Mantenibilidad
- Integración con módulos estándar

5. Diseño del Modelo de Datos

5.1 Modelo Principal: Pistas

Se creó el modelo:

club.padel.pista

Campos implementados:

- name (Char)
- estado (Selection)
- Posibilidad futura de:
 - precio_hora (Float)
 - tipo_pista (Indoor/Outdoor)
 - iluminación (Boolean)

Se diseñó pensando en ampliación sin necesidad de modificar estructura base.

5.2 Modelo de Reservas Implementado

- Se implementó el modelo:
- **club_padel.reserva**
- Campos principales:
- Relación Many2One con pista (club_padel.pista).

- Relación Many2one con cliente (res.partner).
- Fecha y hora de inicio de la reserva.
- Fecha y hora de fin de la reserva.
- Estado de la reserva (borrador, confirmada, cancelada).
- Además, se añadió integración con la tienda mediante el módulo puente, permitiendo asociar carritos de productos a cada reserva sin modificar directamente la estructura base del modelo principal.

6. Seguridad y Permisos

Se definieron reglas de acceso mediante:

`security/ir.model.access.csv`

Permisos diferenciados:

- Usuario básico → lectura
- Administrador → creación y modificación

Esto garantiza:

- Integridad de datos
- Control interno
- Prevención de errores humanos

7. Personalización del Entorno Gráfico

7.1 Gestión de Assets

Se integró CSS personalizado mediante: **`club_padel.css`**

GNU nano 6.2 /opt/odoo/custom_addons/club_padel/static/src/css/club_padel.css *

```
/* =====
TEMA CLUB PÁDEL
===== */

/* Barra superior */
.o_main_navbar {
    background: linear-gradient(90deg, #0f5132, #198754) !important;
    border-bottom: 3px solid #0a3d27;
}

.o_main_navbar .o_menu_brand,
.o_main_navbar .o_menu_sections a {
    color: #ffffff !important;
    font-weight: 600;
}

/* Menú lateral */
.o_web_client .o_menu_apps {
    background-color: #f4f7f5 !important;
}

.o_web_client .o_menu_apps .o_app {
    color: #0f5132 !important;
    font-weight: bold;
}

/* Hover menú lateral */
.o_web_client .o_menu_apps .o_app:hover {
    background-color: #d1e7dd !important;
    color: #0a3d27 !important;
}

/* Tabla estilo deportivo */
.o_list_view thead {
    background-color: #0f5132 !important;
    color: white !important;
}

.o_list_view tbody tr:nth-child(even) {
    background-color: #e9f5ee !important;
}

/* Botón NEW */
.btn-primary {
    background-color: #198754 !important;
    border-color: #0f5132 !important;
    font-weight: bold;
}

.btn-primary:hover {
    background-color: #0f5132 !important;
}

/* Formularios */
.o_form_view .o_form_sheet_bg {
    background: linear-gradient(180deg, #ffffff, #f1f9f4);
}

/* Títulos */
.o_control_panel .breadcrumb {
    color: #0f5132 !important;
    font-weight: bold;
}
```

```
operador_odoo@ldapserver:~/custom_addons$ grep -n "assets" /opt/odoo/custom_addons/  
club_padel/__manifest__.py  
47:     "assets": {  
48:         "web.assets_backend": [  
operador_odoo@ldapserver:~/custom_addons$
```

Se validó carga mediante:

?debug=assets

Se aplicaron cambios visibles:

- Navbar corporativo
- Botones personalizados
- Cabeceras estilizadas
- Fondo corporativo

Justificación técnica:

Se utilizó `assets_backend` para afectar únicamente al entorno administrativo, evitando conflictos con web pública.

8. Metodología de Trabajo

Se siguió una metodología incremental:

1. Análisis de necesidades.
2. Diseño conceptual.
3. Creación estructura módulo.
4. Desarrollo de modelos.
5. Creación vistas.
6. Integración con módulos estándar.
7. Personalización gráfica.
8. Pruebas.
9. Backup final.

Este enfoque permitió detectar errores progresivamente.

9. Pruebas Realizadas

Además de las pruebas iniciales, se realizaron pruebas específicas de integración entre módulos:

- Creación de pistas y reservas funcionales.

- Confirmación de reservas y validación de estados.
- Creación de productos en el módulo tienda_padel.
- Asociación de productos a reservas mediante carritos.
- Cálculo automático de totales en el carrito.
- Verificación de relaciones Many2one y One2many entre módulos.

Estas pruebas garantizaron la correcta interacción entre los tres módulos desarrollados y la estabilidad general del sistema ERP implantado.

```
operador_odoo@ldapserver:~/custom_addons$ sudo -u operador_odoo /usr/bin/python3 /opt/odoo/odoo-bin \
--addons-path=/opt/odoo/addons,/opt/odoo/custom_addons \
-d bd_empresa1 -u club_padel --stop-after-init
operador_odoo@ldapserver:~/custom_addons$
operador_odoo@ldapserver:~/custom_addons$ sudo systemctl restart odoo16
[sudo] contraseña para operador_odoo:
operador_odoo@ldapserver:~/custom_addons$
```

10. Rendimiento y Optimización

El sistema presenta:

- Baja carga inicial
- Arquitectura preparada para crecimiento
- Integración directa con ORM eficiente de Odoo

No se realizaron optimizaciones avanzadas al tratarse de entorno de tamaño medio, pero el sistema permite:

- Indexación futura
- Automatización de tareas programadas
- Uso de workers múltiples

11. Escalabilidad del Proyecto

El sistema está preparado para:

- Implementar reservas online
- Automatizar facturación
- Integrar TPV

- Crear dashboard con KPIs:
 - Ingresos mensuales
 - Ocupación de pistas
 - Ventas por producto
 - Ranking de clientes

12. Plan de Mantenimiento

Se recomienda:

- Backup semanal automático
- Actualización de módulos controlada
- Monitorización del servicio
- Control de logs en:

```
operador_odoo@ldapserver:~$ ls /var/log/odoo/  
log.txt  
operador_odoo@ldapserver:~$
```

13. Valor Técnico del Proyecto

Este proyecto demuestra:

- Capacidad de análisis de necesidades reales.
- Implementación práctica de ERP.
- Desarrollo de módulo propio.
- Uso correcto de herencia de vistas.
- Integración de assets.
- Diagnóstico técnico avanzado.
- Gestión de entorno Linux.
- Comprensión del ORM de Odoo.

14. Conclusión

La implantación realizada no solo ha digitalizado los procesos principales del club, sino que ha construido una arquitectura modular sólida basada en tres módulos propios: club_padel, tienda_padel y club_padel_tienda.

El sistema final permite gestionar reservas deportivas y ventas de productos en un entorno unificado, demostrando una implantación completa y escalable de Odoo en un contexto empresarial real.